Chapter 14

Unsupervised learning

Chapters 2–9 walked through the *supervised learning* pipeline. We defined models that mapped observed data $\mathbf{x}$ to output values $\mathbf{y}$ and introduced loss functions that measured the quality of that mapping for a training dataset $\{\mathbf{x}_i, \mathbf{y}_i\}$. Then we discussed how to fit and measure the performance of these models. Chapters 10–13 introduced more sophisticated model architectures incorporating parameter sharing and allowing parallel computational paths.

The defining characteristic of *unsupervised learning models* is that they are learned from a set of observed data $\{\mathbf{x}_i\}$ in the absence of labels. All unsupervised models share this property, but they have diverse goals. They may be used to generate plausible new samples from the dataset or to manipulate, denoise, interpolate between, or compress examples. They can also be used to reveal the internal structure of a dataset (e.g., by dividing it into coherent clusters) or to distinguish whether new examples belong to the same dataset or are outliers.

This chapter introduces a taxonomy of unsupervised learning models and then discusses the desirable properties of models and how to measure their performance. The four subsequent chapters discuss four particular models: generative adversarial networks (GANs), normalizing flows, variational autoencoders (VAEs), and diffusion models.[1]

14.1 Taxonomy of unsupervised learning models

A common strategy in unsupervised learning is to define a mapping between the data examples $\mathbf{x}$ and a set of unseen *latent* variables $\mathbf{z}$. These latent variables capture underlying structure in the dataset and usually have a lower dimension than the original data; in this sense, a latent variable $\mathbf{z}$ can be considered a compressed version of a data example $\mathbf{x}$ that captures its essential qualities (figures 1.9–1.10).

In principle, the mapping between the observed and latent variables can be in either direction. Some models map from the data $\mathbf{x}$ to latent variables $\mathbf{z}$. For example, the

[1] Until this point, almost all of the relevant math has been embedded in the text. However, the following four chapters require a solid knowledge of probability. Appendix C covers the relevant material.

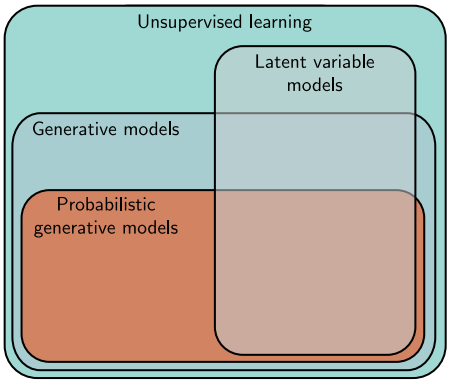

Figure 14.1 Taxonomy of unsupervised learning models. Unsupervised learning refers to any model trained on datasets without labels. Generative models can synthesize (generate) new examples with similar statistics to the training data. A subset of these are probabilistic and define a distribution over the data. We draw samples from this distribution to generate new examples. Latent variable models define a mapping between an underlying explanatory (latent) variable and the data and may fall into any of the above categories.

famous *k-means* algorithm maps the data $\mathbf{x}$ to a cluster assignment $z \in \{1, 2, \ldots, K\}$. Other models map from the latent variables $\mathbf{z}$ to the data $\mathbf{x}$. Consider defining a distribution $Pr(\mathbf{z})$ over the latent variable $\mathbf{z}$ in these models. New examples can now be *generated* by (i) drawing from this distribution and (ii) mapping the sample to the data space $\mathbf{x}$. Accordingly, these are termed *generative models* (see figure 14.1).

The four models in chapters 15 to 18 are all generative models that use latent variables. *Generative adversarial networks* (chapter 15) learn to generate data examples $\mathbf{x}^*$ from latent variables $\mathbf{z}$, using a loss that encourages the generated samples to be indistinguishable from real examples (figure 14.2a).

Normalizing flows, *variational autoencoders*, and *diffusion models* (chapters 16–18) are *probabilistic generative models*. In addition to generating new examples, they assign a probability $Pr(\mathbf{x}|\boldsymbol{\phi})$ to each data point $\mathbf{x}$. This will depend on the model parameters $\boldsymbol{\phi}$, and in training, we maximize the probability of the observed data $\{\mathbf{x}_i\}$, so the loss is the sum of the negative log-likelihoods (figure 14.2b):

$$L[\boldsymbol{\phi}] = -\sum_{i=1}^{I} \log\Big[Pr(\mathbf{x}_i|\boldsymbol{\phi})\Big]. \tag{14.1}$$

Since probability distributions must sum to one, this implicitly reduces the probability of examples that lie far from the observed data. As well as providing a training criterion, assigning probabilities is useful in its own right; the probability on a test set can be used to compare two models quantitatively, and the probability for an example can be thresholded to determine if it belongs to the same dataset or is an *outlier*.[2]

14.2 What makes a good generative model?

Generative models based on latent variables should have the following properties:

[2]Note that not all probabilistic generative models rely on latent variables. The transformer decoder (section 12.7) was learned without labels, can generate new examples, and can assign a probability to these examples but is based on an autoregressive formulation (equation 12.15).

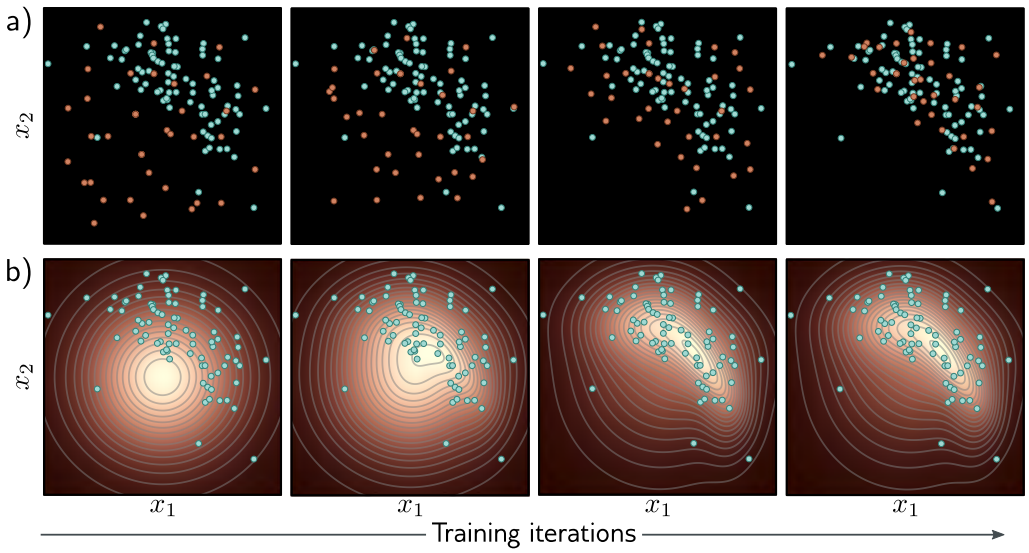

Figure 14.2 Fitting generative models a) Generative adversarial models provide a mechanism for generating samples (orange points). As training proceeds (left to right), the loss function encourages these samples to become progressively less distinguishable from real examples (cyan points). b) Probabilistic models (including variational autoencoders, normalizing flows, and diffusion models) learn a probability distribution over the training data. As training proceeds (left to right), the likelihood of the real examples increases under this distribution, which can be used to draw new samples and assess the probability of new data points.

- **Efficient sampling:** Generating samples from the model should be computationally inexpensive and take advantage of the parallelism of modern hardware.
- **High-quality sampling:** The samples should be indistinguishable from the real data with which the model was trained.
- **Coverage:** Samples should represent the entire training distribution. It is insufficient to generate samples that all look like a subset of the training examples.
- **Well-behaved latent space:** Every latent variable $\mathbf{z}$ corresponds to a plausible data example $\mathbf{x}$. Smooth changes in $\mathbf{z}$ correspond to smooth changes in $\mathbf{x}$.
- **Disentangled latent space:** Manipulating each dimension of $\mathbf{z}$ should correspond to changing an interpretable property of the data. For example, in a model of language, it might change the topic, tense, or verbosity.
- **Efficient likelihood computation:** If the model is probabilistic, we would like to be able to calculate the probability of new examples efficiently and accurately.

This naturally leads to the question of whether the generative models that we consider satisfy these properties. The answer is subjective, but figure 14.3 provides guidance. The precise assignments are disputable, but most practitioners would agree that there is no single model that satisfies all of these characteristics.

Model	Efficient	Sample quality	Coverage	Well-behaved latent space	Disentangled latent space	Efficient likelihood
GANs	✓	✓	✗	✓	?	n/a
VAEs	✓	✗	?	✓	?	✗
Flows	✓	✗	?	✓	?	✓
Diffusion	✗	✓	?	✗	✗	✗

Figure 14.3 Properties of four generative models. Neither generative adversarial networks (GANs), variational autoencoders (VAEs), normalizing flows (Flows), nor diffusion models (diffusion) have the full complement of desirable properties.

14.3 Quantifying performance

The previous section discussed the desirable properties of generative models. We now consider quantitative measures of success for generative models. Much experimentation with generative models has used images due to the widespread availability of that data and the ease of qualitatively judging the samples. Consequently, some of these metrics only apply to images.

Test likelihood: One way to compare probabilistic models is to measure their likelihood for a test dataset. It is ineffective to measure the training data likelihood because a model could assign a very high probability to each training point and very low probabilities in between. This model would have a very high training likelihood but could only reproduce the training data. The test likelihood captures how well the model generalizes from the training data and also the coverage; if the model assigns a high probability to just a subset of the training data, it must assign lower probabilities elsewhere, so a portion of the test examples will have low probability.

Test likelihood is a sensible way to quantify probabilistic models, but unfortunately, it is not relevant for generative adversarial models (which do not assign a probability) and is expensive to estimate for variational autoencoders and diffusion models (although it is possible to compute a lower bound on the log-likelihood). Normalizing flows are the only type of model for which the likelihood can be computed exactly and efficiently.

Inception score: The inception score (IS) is specialized for images and ideally for generative models trained on the ImageNet database. The score is calculated using a pretrained classification model – usually the "Inception" model, from which the name is derived. It is based on two criteria. First, each generated image $\mathbf{x}^*$ should look like one and only one of the 1000 possible classes y in the ImageNet database. Hence, the probability distribution $Pr(y|\mathbf{x}_i^*)$ should be highly peaked at the correct class. Second, the entire set of generated images should be assigned to the classes with equal probability, so $Pr(y)$ should be flat when averaged over all generated examples.

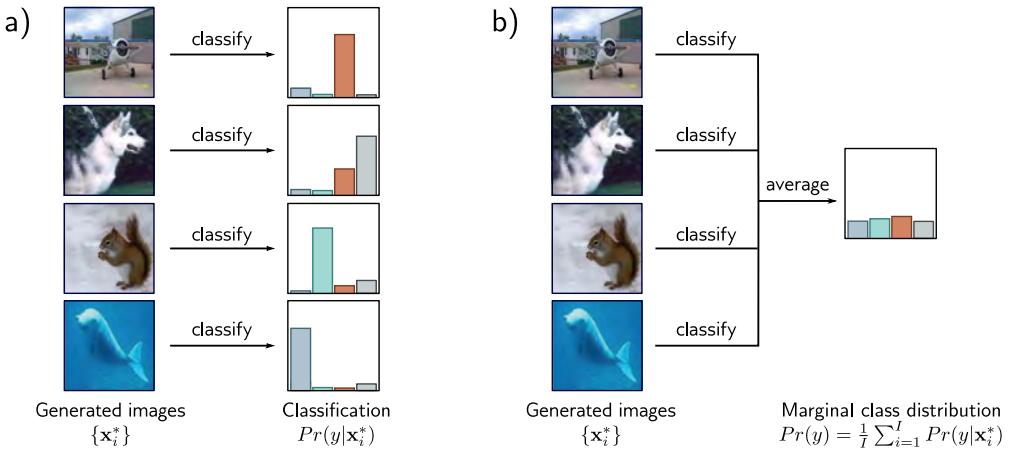

Figure 14.4 Inception score. a) A pretrained network classifies the generated images. If the images are realistic, the resulting class probabilities $Pr(y|\mathbf{x}_i^*)$ should be peaked at the correct class. b) If the model generates all classes equally frequently, the marginal (average) class probabilities should be flat. The inception score measures the average distance between the distributions in (a) and the distribution in (b). Images from Deng et al. (2009).

The inception score measures the average distance between these two distributions over the generated set. This distance will be large if one is peaked and the other flat (figure 14.4). More precisely, it returns the exponential of the expected KL-divergence between $Pr(y|\mathbf{x}_i^*)$ and $Pr(y)$:

Appendix C.5.1 KL divergence

$$IS = \exp\left[\frac{1}{I}\sum_{i=1}^{I} D_{KL}\Big[Pr(y|\mathbf{x}_i^*)||Pr(y)\Big]\right], \tag{14.2}$$

where I is the number of generated examples and:

$$Pr(y) = \frac{1}{I}\sum_{i=1}^{I} Pr(y|\mathbf{x}_i^*). \tag{14.3}$$

This metric is only sensible for generative models of the ImageNet database and is sensitive to the particular classification model; retraining this model can give quite different numerical results. Moreover, it does not reward diversity within an object class; it returns a high value if the model only generates one realistic example of each class.

Fréchet inception distance: This measure is also intended for images and computes a symmetric distance between the distributions of generated samples and real examples. This must be approximate since it is hard to characterize either distribution (indeed,

characterizing the distribution of real examples is the job of generative models in the first place). Hence, the Fréchet inception distance approximates both distributions by multivariate Gaussians and (as the name suggests) estimates the distance between them using the Fréchet distance.

Appendix C.5.4 Fréchet distance

However, it does not model the distance with respect to the original data but rather the activations in the deepest layer of the inception classification network. These hidden units are the ones most associated with object classes, so the comparison occurs at a semantic level, ignoring the more fine-grained details of the images. This metric does take account of diversity within classes but relies heavily on the information retained by the features in the inception network; any information discarded by the network does not contribute to the result. Some of this discarded information may still be important to generate realistic samples.

Manifold precision/recall: Fréchet inception distance is sensitive both to the realism of the samples and their diversity but does not distinguish between these factors. To disentangle these qualities, we consider the overlap between the data *manifold* (i.e., the subset of the data space where the real examples lie) and the model manifold (i.e., where the generated samples lie). The *precision* is the fraction of model samples that fall into the data manifold. This measures the proportion of generated samples that are realistic. The *recall* is the fraction of data examples that fall within the model manifold. This measures the proportion of the real data the model can generate (figure 14.5).

To estimate the manifold, we place a hypersphere around each data example, whose radius is the distance to the k^{th} nearest neighbor. The union of these spheres is an approximation of the manifold, and it's easy to determine if a new point lies within it. This manifold is also typically computed in the feature space of a classifier with the advantages and disadvantages that entails.

14.4 Summary

Unsupervised models learn about the structure of a dataset in the absence of labels. A subset of these models is generative and can synthesize new data examples. A further subset is probabilistic in that they can both generate new examples and assign a probability to observed data. The models considered in the following four chapters start with a latent variable $\mathbf{z}$ which has a known distribution. A deep neural network then maps from the latent variable to the observed data space. We considered desirable properties of generative models and introduced metrics that attempt to quantify their performance.

Notes

Popular generative models include generative adversarial networks (Goodfellow et al., 2014), variational autoencoders (Kingma & Welling, 2014), normalizing flows (Rezende & Mohamed,

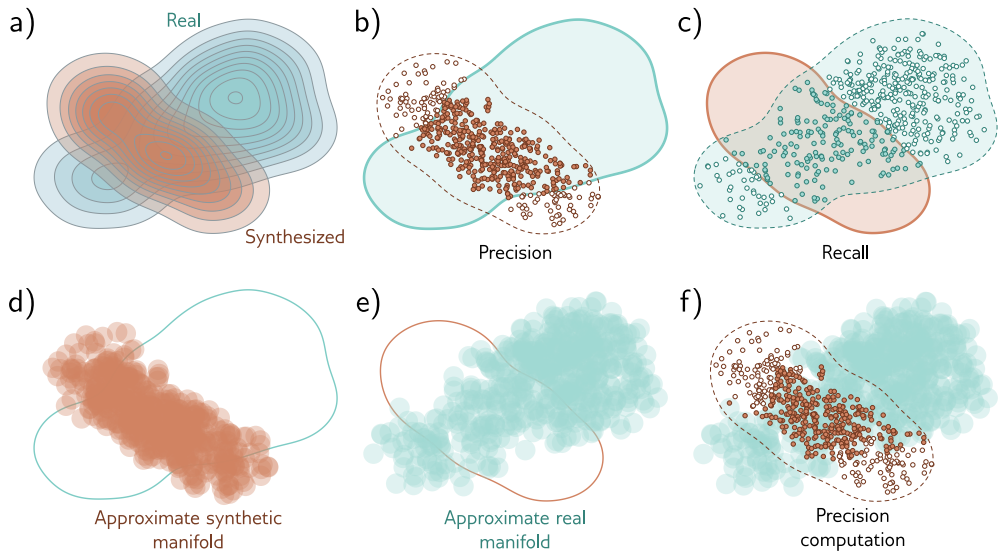

Figure 14.5 Manifold precision/recall. a) True distributions of real examples and samples synthesized by the generative model. b) The overlap can be summarized by the *precision* (the proportion of synthesized samples that overlap with the distribution or *manifold* of real examples), and c) *recall* (the proportion of real examples that overlap with the manifold of the synthesized samples). d) The manifold of synthesized samples can be approximated by taking the union of a set of hyperspheres centered on each sample. Here, these have constant radius, but more commonly, the radius is based on the distance to the k^{th} nearest neighbor. e) The manifold for real examples is approximated similarly. f) The precision can be computed as the proportion of samples that lie within the approximated manifold of real examples. Similarly, the recall is computed as the proportion of real examples that lie within the approximated manifold of samples (not shown). Adapted from Kynkäänniemi et al. (2019).

2015), diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020), autoregressive models (Bengio et al., 2000; Van den Oord et al., 2016b), and energy-based models (LeCun et al., 2006). All except energy models are discussed in this book. Bond-Taylor et al. (2022) provide a recent survey of generative models.

Evaluation: Salimans et al. (2016) introduced the inception score, and Heusel et al. (2017) introduced the Fréchet inception distance, both of which are based on the Pool-3 layer of the Inception V3 model (Szegedy et al., 2016). Nash et al. (2021) used earlier layers of the same network that retain more spatial information to ensure that the spatial statistics of images are also replicated. Kynkäänniemi et al. (2019) introduced the manifold precision/recall method. Barratt & Sharma (2018) discuss the inception score in detail and point out its weaknesses. Borji (2022) discusses the pros and cons of different methods for assessing generative models.